# R documentation

of all in 'man'

September 22, 2016

## R topics documented:


baserateCT . . . 1
baserateSet . . . 2
codeSet . . . 3
contingencyTable . . . 3
createSimulatedCodeSet . . . 4
getTestSet . . . 4
kappaCT . . . 5
kappaSet . . . 6
rho . . . 6
rhoCT . . . 8
rhoMin . . . 9
rhoR . . . 9
rhoSet . . . 10

**Index** **12**


---

`baserateCT` *Calculate Baserate (CT)*

---

### Description

This function calculates the baserate of the first rater, second rater, and the average of both the raters.

### Usage

```
baserateCT(CT)
```

### Arguments

| | |
|---|---|
| `CT` | The `contingencyTable` for which the baserate is calculated |

### Details

A baserate is the percentage, as a decimal, that a positive code appears in data (either a codeSet or contingencyTable) for a given rater. It is assumed that the first rater is more experienced and thus provides a better estimation of the actual baserate for a given code, so the first rater's baserate is often used as if it is the actual baserate. If the raters are assumed to have the same experience level, the average baserate may give a better estimation. If the second rater is more experienced, the second rater's baserate may give a better estimation. Functions assume that the first rater is the more experienced rater and thus uses the first rater's baserate as the overall baserate estimation.

### Value

A list of the format:

**firstBaserate** The percentage of the data for which a positive code, or a 1, appears in the first rater

**secondBaserate** The percentage of the data for which a positive code, or a 1, appears in the second rater

**averageBaserate** The average of the firstBaserate and secondBaserate.

### See Also

baserateSet

---

## baserateSet *Calculate Baserate (Set)*

---

### Description

This function calculates the baserate of the first rater, second rater, and the average of both the raters. Called by baserate.

### Usage

```
baserateSet(set)
```

### Arguments

| | |
|---|---|
| set | The codeSet for which the baserate is calculated |

### Value

A list of the format:

**firstBaserate** The percentage that a positive code, or a 1, appears in the first rater

**secondBaserate** The percentage that a positive code, or a 1, appears in the second rater

**averageBaserate** The average percentage that a positive code, or a 1, appears in either of the two raters

### See Also

baserateCT

codeSet *codeSet*

## Description

A codeSet is a Nx2 binary matrix in which the first column corresponds to the first rater and the second column corresponds to the second rater.

## Usage

```
codeSet
```

## Format

The codeSet is an object of class matrix with n rows and two columns.

## Examples

```
#An example codeSet
firstRater = c(1,1,1,1,rep(0,36))
secondRater = c(1,1,1,0,1,1,rep(0,34))
exampleSet = cbind(firstRater,secondRater)

#This set is included in the package under the variable name "codeSet".
```

contingencyTable *contingencyTable*

## Description

A contingency Table is a 2x2 matrix that contains the counts of all combinations of positive and negative ratings made by two raters.

## Usage

```
contingencyTable
```

## Format

The contingency Table is an object of class matrix with two rows and two columns. The ordering of the combination vector input to the matrix is as follows: c(Rater1Positive & Rater2Positive, Rater1Negative & Rater2Positive, Rater1Positive & Rater2Negative, Rater1Negative & Rater2Negative).

## Examples

```
#A example contingencyTable
ct = matrix(c(3,2,1,34), nrow = 2, ncol = 2)

#This contingencyTable is included in the package under the variable name "contingencyTable".
```

createSimulatedCodeSet *Create Simulated codeSet*

## Description

Creates a simulated codeSet with the given parameters

## Usage

```
createSimulatedCodeSet(length, baserate, kappaMin, kappaMax, precisionMin,
  precisionMax)
```

## Arguments

| | |
|---|---|
| length | the length of the simulated codeSet to be created |
| baserate | the baserate of the simulated codeSet |
| kappaMin | the minimum kappa of the simulated codeSet |
| kappaMax | the maximum kappa of the simulated codeSet |
| precisionMin | the minimum precision of the simulated codeSet |
| precisionMax | the maximum precision of the simulated codeSet |

## Details

codeSets are generated by first picking a random kappa within its range and a random precision within its range. If the random kappa, random precision, and baserate are not mathematically possible, then the precision is resampled from a range of mathematically possible values within its range. A unique simulated codeSet is then constructed given these parameters.

## Value

A codeSet that fulfills the given parameters

getTestSet *Get Test Set*

## Description

This function gets a *testSet* from a larger codeSet given certain sampling parameters.

## Usage

```
getTestSet(set, testSetLength, testSetBaserateInflation)
```

## Arguments

| | |
|---|---|
| set | The codeSet from which the *testSet* is taken |
| testSetLength | The length of the *testSet* to be taken |
| testSetBaserateInflation | The minimum guaranteed baserate of the *testSet* |

### Details

A *testSet* is a codeSet that is a subset of a larger codeSet with a given set of properties. A *testSet* is constructed by sampling (without replacement) P rows from rows in the larger codeSet where the first rater's code was 1, and then appending an additional sample (without replacement) of R rows taken at random from the larger codeSet excluding rows included in the first P rows sampled. P is computed as the minbasetate * length of the *testset*. R is computed as testSetLength - P. THe result of this sampling procedure is to create a sample with a minimum baserate of baserate regardless of the baserate of the larger codeSet.

### Value

A codeSet with the properties specified

---

## kappaCT *Calculate kappa (contingency Table)*

---

### Description

This function calculates Cohen's kappa on a contingencyTable or a codeSet.

### Usage

```
kappaCT(ct)
```

### Arguments

| | |
|---|---|
| data | A contingencyTable or a codeSet |

### Value

The kappa of the contingencyTable or codeSet

### See Also

kappaSet and kappaCT

### Examples

```
#Given a contingency Table
kappaCT(contingencyTable)

#Given a code set
kappaCT(codeSet)
```

## kappaSet *Calculate kappa (Set)*

### Description

This function calculates Cohen's kappa for a given codeSet. Called by kappa.

### Usage

```
kappaSet(set)
```

### Arguments

| | |
|---|---|
| set | A codeSet |

### Value

The kappa of the codeSet

## rho *Rho*

### Description

This function calculates rho for a testSet, contingencyTable, or an observed kappa value with associated set parameters (testSetLength and OCSBaserate).

### Usage

```
rho(observedKappa, OcSBaserate, testSetLength, testSetBaserateInflation = 0,
  OcSLength = 10000, replicates = 800, ScSKappaThreshold = 0.65,
  ScSKappaMin = 0.4, ScSPrecisionMin = 0.6, ScSPrecisionMax = 1)
```

### Arguments

| | |
|---|---|
| OcSBaserate | The baserate of the observed codeSet (defaults to baserate of testSet or contingencyTable) |
| testSetLength | The length of the testSet (ignored unless *data* is an observed kappa value) |
| testSetBaserateInflation | The minimum baserate from the sampling procedure |
| OcSLength | The length of the observed codeSet |
| replicates | The number of simulated codeSets to use in the null hypothesis distribution for rho; similar to replicates in a Monte Carlo study |
| ScSKappaThreshold | The maximum kappa value used to generate simulated codeSets in the null hypothesis distribution for rho |
| ScSKappaMin | The minimum kappa value used to generate simulated codeSets in the null hypothesis distribution for rho |

ScSPrecisionMin
: The minimum precision to be used for generation of simulated codeSets in the null hypothesis distribution for rho

ScSPrecisionMax
: The maximum precision to be used for generation of simulated codeSets in the null hypothesis distribution for rho

data
: The observed kappa value, testSet or contingecyTable that will be tested with rho

## Details

Rho is a monte carlo rejective method of interrater reliability statistics, implemented here for Cohen's Kappa. Rho constructs a collection of data sets in which kappa is below a specified threshold, and computes the empirical distribution on kappa based on the specified sampling procedure. Rho returns the percent of the empirical distribution greater than or equal to an observed kappa. As a result, Rho quantifies the type 1 error in generalizing from an observed test set to a true value of agreement between two raters.

Rho starts with an observed kappa value, calculated on a subset of a codeSet, known as an observed testSet, and a *kappa threshold* which indicates what is considered significant agreement between raters.

It then generates a collection of fully-coded, simulated codeSets (ScS), further described in createScS, all of which have a kappa value below the kappa threshold and similar properties as the original codeSet.

Then, kappa is calculated on a testSet sampled from each of the ScSs in the collection to create a null hypothesis distribution. These testSets mirror the observed testSet in their size and sampling method. How these testSets are sampled is futher described in testSet

The null hypothesis is that the observed testSet, was sampled from a data set, which, if both raters were to code in its entirety, would result in a level of agreement below the kappa threshold.

For example, using an alpha level of 0.05, if the observed kappa is greater than 95 percent of the kappas in the null hypothesis distribution, the null hypothesis is rejected. Then one can conclude that the two raters would have acceptable agreement had they coded the entire data set.

## Value

rho for the given parameters

## Examples

```
# Given only required parameters
rho(observedKappa = 0.88, OcSBaserate = 0.2, testSetLength = 80)

# Given required parameters plus testSetBaserateInflation and runs
rho(
 observedKappa = 0.88, OcSBaserate = 0.2, testSetLength = 80,
 testSetBaserateInflation = 0.2, replicates = 500
)
```

rhoCT *Rho (contingency Table)*

## Description

This function calculates rho and kappa for a given contingencyTable, and returns a list containing both values. Called by rho.

## Usage

```
rhoCT(contingencyTable, OcSBaserate = NULL, testSetBaserateInflation = 0,
  OcSLength = 10000, replicates = 800, ScSKappaThreshold = 0.65,
  ScSKappaMin = 0.4, ScSPrecisionMin = 0.6, ScSPrecisionMax = 1)
```

## Arguments

| | |
|---|---|
| contingencyTable | The contingencyTable used to calculate rho |
| OcSBaserate | The baserate of the observed codeSet |
| testSetBaserateInflation | The minimum baserate from the sampling procedure |
| OcSLength | The length of the observed codeSet |
| replicates | The number of simulated codeSets to use in the null hypothesis distribution for rho; similar to replicates in a Monte Carlo study |
| ScSKappaThreshold | The maximum kappa value used to generate simulated codeSets in the null hypothesis distribution for rho |
| ScSKappaMin | The minimum kappa value used to generate simulated codeSets in the null hypothesis distribution for rho |
| ScSPrecisionMin | The minimum precision to be used for generation of simulated codeSets in the null hypothesis distribution for rho |
| ScSPrecisionMax | The maximum precision to be used for generation of simulated codeSets in the null hypothesis distribution for rho |

## Value

A list of the format:

**rho** The rho of the contingencyTable

**kappa** The Cohen's Kappa of the contingencyTable

## Examples

```
# Given testSetBaserateInflation
rhoCT(contingencyTable, testSetBaserateInflation = 0.2)

# Given ScSBaserate and testSetBaserateInflation
rhoCT(contingencyTable, OcSBaserate = 0.1, testSetBaserateInflation = 0.2)
```

rhoMin *Rho Min testSetLength*

## Description

This function calculates the minimum testSetLength where it is possible to get a rho less than alpha for the given parameters of rho.

## Usage

```
rhoMin(baserate, alpha = 0.05, inc = 10, printInc = FALSE, ...)
```

## Arguments

| | |
|---|---|
| baserate | A baserate |
| alpha | The threshold of significance for rho (similar to an alpha level for a p value), defaulted to 0.05 |
| inc | An integer indicating by how much the testSetLength should increase each iteration |
| printInc | A boolean indicating whether to print out each increment value with it's corresponding significance for rho |
| ... | Any additional parameters passed into rho |

## Value

The minimum length of testSet, as a denomination of inc, that would give a value where rho becomes mathematically possible

## Examples

```
#Add testSetBaserateInflation as an additional parameter
rhoMinTSLength(0.2, testSetBaserateInflation = 0.33)

#Add testSetBaserateInflation as well as changing inc and selecting printInc
rhoMinTSLength(0.2, inc = 5, printInc = TRUE, testSetBaserateInflation = 0.33)
```

rhoR *rhoR: A package for computing rho.*

## Description

rho is used to test the generalization of inter rater reliability (IRR) statistics, in this case Cohen's Kappa.

## Details

Rho is a monte carlo rejective method of interrater reliability statistics, implemented here for Cohen's Kappa. Rho constructs a collection of data sets in which kappa is below a specified threshold, and computes the empirical distribution on kappa based on the specified sampling procedure. Rho returns the percent of the empirical distribution greater than or equal to an observed kappa. As a result, Rho quantifies the type 1 error in generalizing from an observed test set to a true value of agreement between two raters.

Rho starts with an observed kappa value, calculated on a subset of a codeSet, known as an observed testSet, and a *kappa threshold* which indicates what is considered significant agreement between raters.

It then generates a collection of fully-coded, simulated codeSets (ScS), further described in createScS, all of which have a kappa value below the kappa threshold and similar properties as the original codeSet.

Then, kappa is calculated on a testSet sampled from each of the ScSs in the collection to create a null hypothesis distribution. These testSets mirror the observed testSet in their size and sampling method. How these testSets are sampled is futher described in testSet

The null hypothesis is that the observed testSet, was sampled from a data set, which, if both raters were to code in its entirety, would result in a level of agreement below the kappa threshold.

For example, using an alpha level of 0.05, if the observed kappa is greater than 95 percent of the kappas in the null hypothesis distribution, the null hypothesis is rejected. Then one can conclude that the two raters would have acceptable agreement had they coded the entire data set.

### rho

Use rhoSet
rhoCT

### kappa

Use kappaSet
kappaCT

### rhoMinTSLength

Use rhoMinTSLength

---

`rhoSet` *Rho (set)*

---

## Description

This function calculates rho and kappa for a given set, and returns a list containing both values. Called by rho.

## Usage

```
rhoSet(set, OcSBaserate = NULL, testSetBaserateInflation = 0,
  OcSLength = 10000, replicates = 800, ScSKappaThreshold = 0.65,
  ScSKappaMin = 0.4, ScSPrecisionMin = 0.6, ScSPrecisionMax = 1)
```

## Arguments

| | |
|---|---|
| set | The test codeSet used to calculate rho |
| OcSBaserate | The baserate of the observed codeSet |
| testSetBaserateInflation | The minimum baserate from the sampling procedure |
| OcSLength | The length of the observed codeSet |
| replicates | The number of simulated codeSets to use in the null hypothesis distribution for rho; similar to replicates in a Monte Carlo study |
| ScSKappaThreshold | The maximum kappa value used to generate simulated codeSets in the null hypothesis distribution for rho |
| ScSKappaMin | The minimum kappa value used to generate simulated codeSets in the null hypothesis distribution for rho |
| ScSPrecisionMin | The minimum precision to be used for generation of simulated codeSets in the null hypothesis distribution for rho |
| ScSPrecisionMax | The maximum precision to be used for generation of simulated codeSets in the null hypothesis distribution for rho |

## Value

A list of the format:

**rho** The rho of the codeSet

**kappa** The Cohen's Kappa of the codeSet

# Index


∗Topic **baserateCT**
baserateCT, 1
∗Topic **baserateSet**
baserateSet, 2
∗Topic **codeSet,**
createSimulatedCodeSet, 4
∗Topic **create**
createSimulatedCodeSet, 4
∗Topic **datasets**
codeSet, 3
contingencyTable, 3
∗Topic **kappaCT**
kappaCT, 5
∗Topic **kapppaSet**
kappaSet, 6
∗Topic **rhoCT**
rhoCT, 8
∗Topic **rhoMinTSLength**
rhoMin, 9
∗Topic **rhoSet**
rhoSet, 10
∗Topic **rho**
rho, 6
∗Topic **testSet**
getTestSet, 4

baserate, *2*, *4*, *6*, *8*, *9*, *11*
baserateCT, 1, *2*
baserateSet, *2*, 2

codeSet, *2*, 3, *4–8*, *10*, *11*
contingecyTable, *7*
contingencyTable, *1*, *2*, 3, *5*, *6*, *8*
createScS, *7*, *10*
createSimulatedCodeSet, 4

getTestSet, 4

kappa, *6*
kappaCT, *5*, 5, *10*
kappaSet, *5*, 6, *10*

rho, 6, *8–10*
rhoCT, 8, *10*
rhoMin, 9
rhoMinTSLength, *10*
rhoR, 9
rhoR-package *(*rhoR*)*, 9
rhoSet, *10*, 10

testSet, *6*, *7*, *10*